

Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

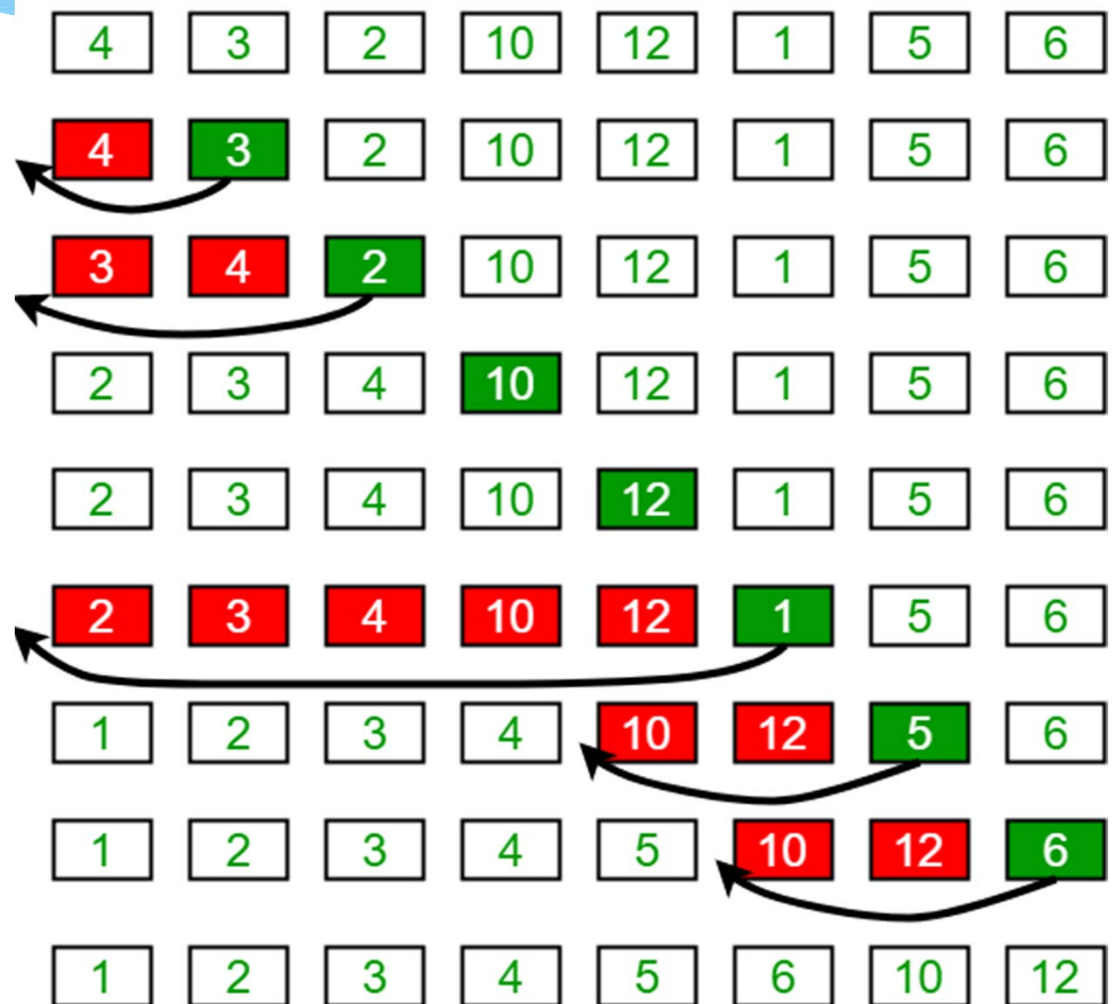
1. Insertion sort,
2. Shellsort,
3. Tree sort

Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Insertion sort
(сортиране чрез вмъкване) е прост алгоритъм за сортиране, който изгражда крайния сортиран масив (или списък) един по един елемент чрез сравнения.

Той е много по-малко ефективен при големи списъци в сравнение с по-усъвършенстваните алгоритми като бързо сортиране или сортиране чрез сливане.

Insertion Sort Execution Example



Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Сортирането чрез вмъкване обаче осигурява няколко предимства:

1. Проста реализация
2. Ефективен за (доста) малки набори от данни, подобно на други квадратични (т.е. $O(n^2)$) алгоритми за сортиране
3. Адаптивен, т.е. ефективен за набори от данни, които вече са значително сортирани: времевата сложност е $O(kn)$, когато всеки елемент във входа е на не повече от k места далеч от неговата сортирана позиция
4. Стабилен; т.е. не променя относителния ред на елементи с равни индекси
5. На място; т.е. изисква само постоянно количество $O(1)$ допълнително пространство в паметта

Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Псевдокод:

```
procedure insertionSort(A: list of sortable items)
```

```
  n = length(A)
```

```
  for i = 1 to n - 1 do
```

```
    j = i
```

```
    while j > 0 and A[j-1] > A[j] do
```

```
      swap(A[j], A[j-1])
```

```
      j = j - 1
```

```
    end while
```

```
  end for
```

```
end procedure
```

[Insertion Sort](#)

Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Shellsort е алгоритъм за сортиране, който е високоефективен и се основава на алгоритъма за сортиране чрез вмъкване. Този алгоритъм избягва големи отмествания, както е при сортиране чрез вмъкване, както е в случаите когато по-малка стойност е най-вдясно и трябва да бъде преместена най-вляво.

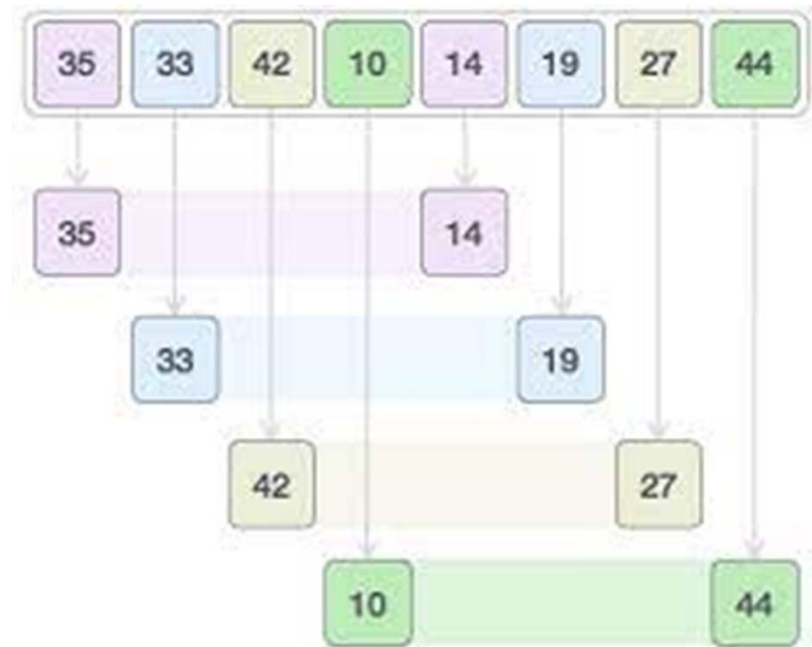
Shellsort

Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Shellsort - Псевдокод

shell_sort (A, N)
where A – list to be sorted; N – gap_size

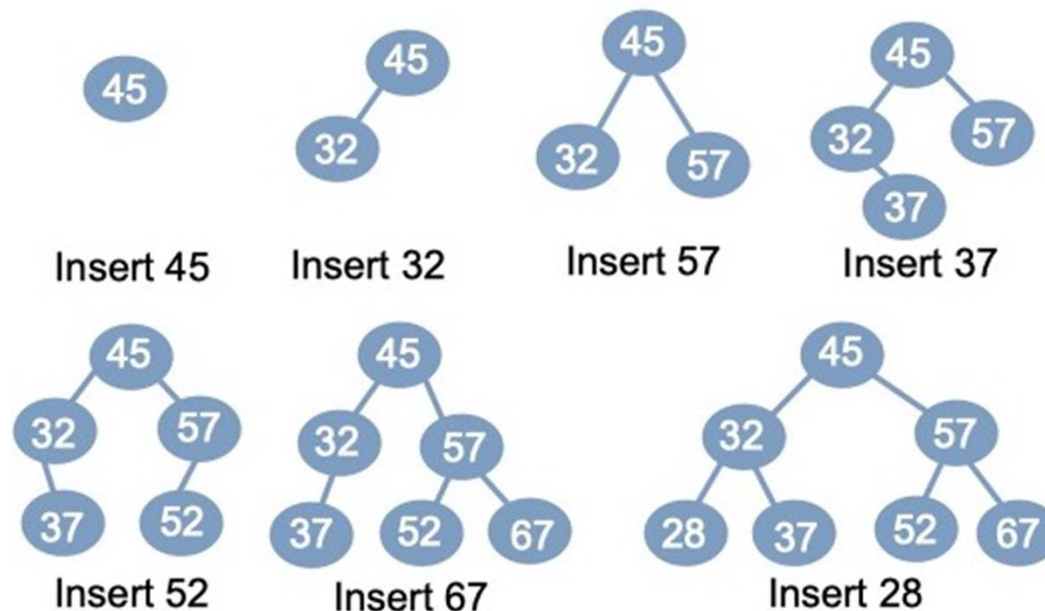
```
set gap_size = N, flag = 1
while gap_size > 1 or flag = 1, repeat
begin
set flag = 0
set gap_size = (gap_size + 1)/2
end
for i = 0 to i < (N-gap_size) repeat
begin
if A[i + gap_size] > A[i]
swap A[i + gap_size], A[i]
set flag = 0
end
end
```



Тема 9 – Алгоритми за сортиране чрез вмъкване (*Insertion sorts*)

Tree sort (сортирането по дърво) е алгоритъм за сортиране, който изгражда двоично дърво за търсене от елементите, които ще бъдат сортирани, и след това преминава през дървото, така че елементите да излязат в сортиран ред.

Tree Sort



[Tree sort](#)

Тема 9 – Алгоритми за сортиране чрез сливане и Алгоритми за сортиране чрез разпределение

1. Алгоритми за сортиране чрез сливане - **Merge sort**
2. Алгоритми за сортиране чрез разпределение (**Distribution sorts**):
Bucket sort, Proxmap sort, Radix sort

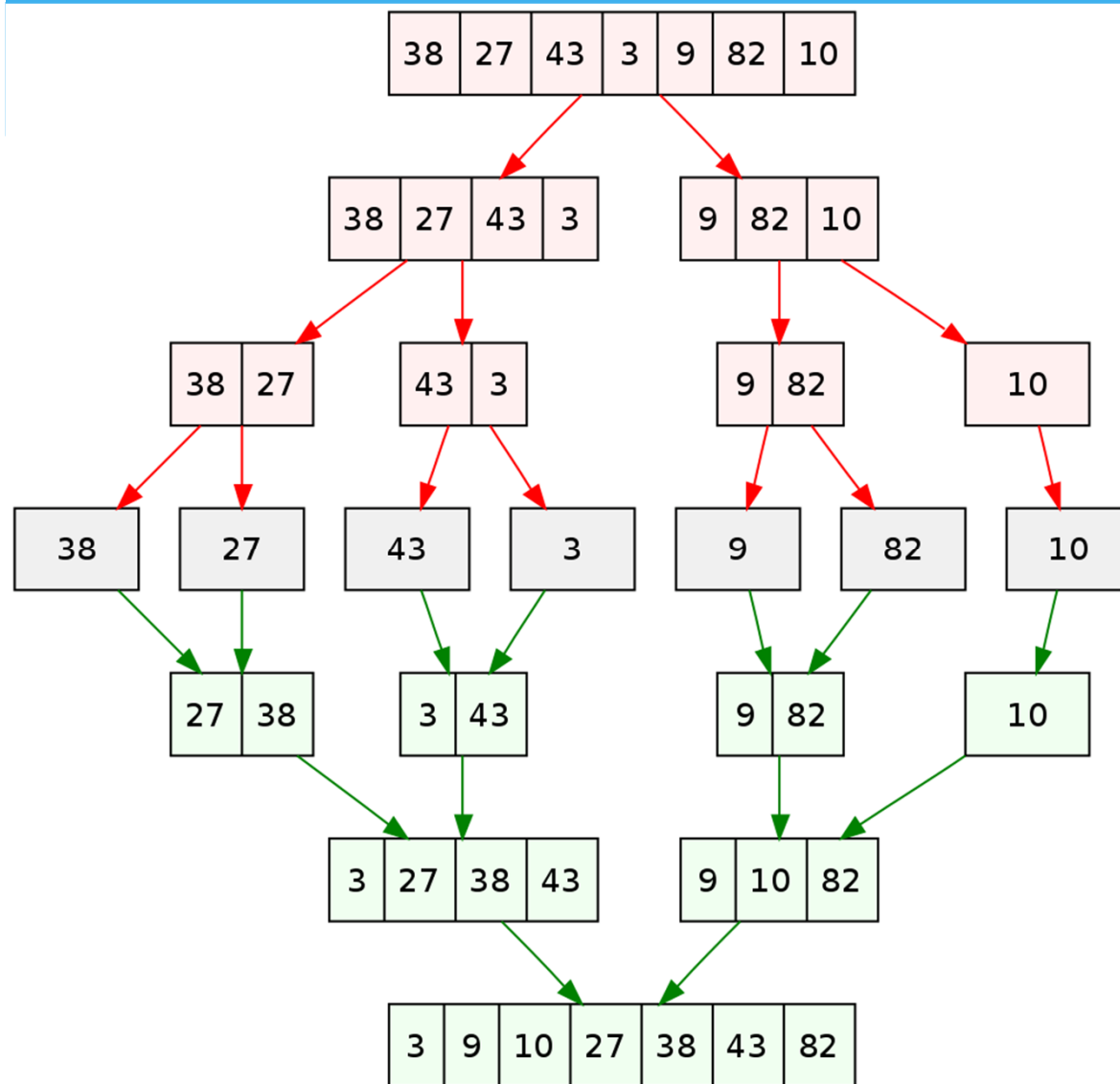
Тема 9 – Алгоритми за сортиране чрез сливане

В компютърните науки **сортирането чрез сливане (Merge sort)** е ефективен алгоритъм за сортиране с общо предназначение и базиран на сравнение. Повечето реализации създават стабилно сортиране, което означава, че редът на равните елементи е еднакъв във входа и изхода.

Концептуално сортирането чрез сливане работи по следния начин:

1. Разделяме несортирания списък на n подсписъци, всеки от които съдържа един елемент (списък от един елемент се счита за сортиран).
2. Многократно обединяваме подсписъци, за да създадем нови сортирани подсписъци, докато остане само един подсписък. Това ще бъде сортираният списък.

Тема 9 – Алгоритми за сортиране чрез сливане



Merge sort

При сортиране на списък от n елемента, сортирането чрез сливане има времева сложност от $O(n \log n)$.

Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)

Bucket sort или **bin sort** е алгоритъм за сортиране, който работи, като разпределя елементите на масива в определен брой групи «**кофи**». След това всяка група се сортира поотделно, като се използва различен алгоритъм за сортиране или чрез рекурсивно прилагане на самия алгоритъм.

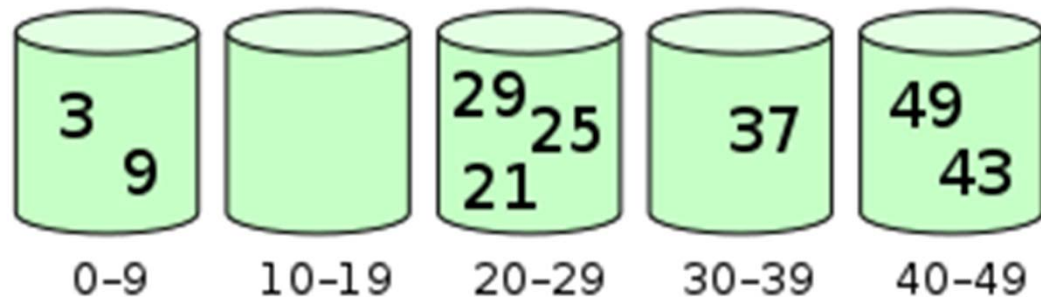
Това е сортиране с разпределение, което се реализира със сравнения и следователно може да се счита и за алгоритъм за сортиране със сравнение. Изчислителната сложност зависи от алгоритъма, използван за сортиране на всяка група «**кофа**», като и от броя на кофите, които се използват, и дали входът е равномерно разпределен.

Bucket sort работи по следния начин:

1. Имаме масив от първоначално празни „кофи“.
2. Елементите от основния масив се разпределят в „кофите“.
3. Всяка от „кофите“ се сортира.
4. Събиране: Посетавят се „кофите“ по ред и всички елементи се връщат обратно в оригиналния масив.

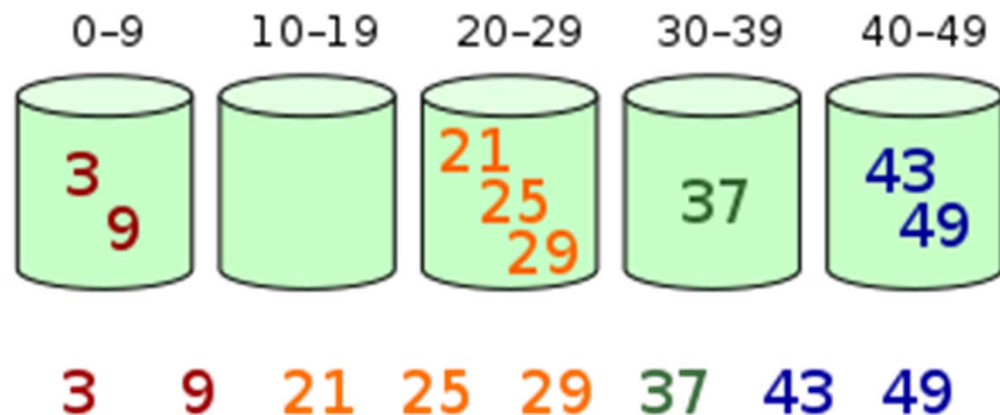
Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)

29 25 3 49 9 37 21 43



Елементите се разпределят между кофите

След края на разпределението, елементите се сортират във всяка кофа.



Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)

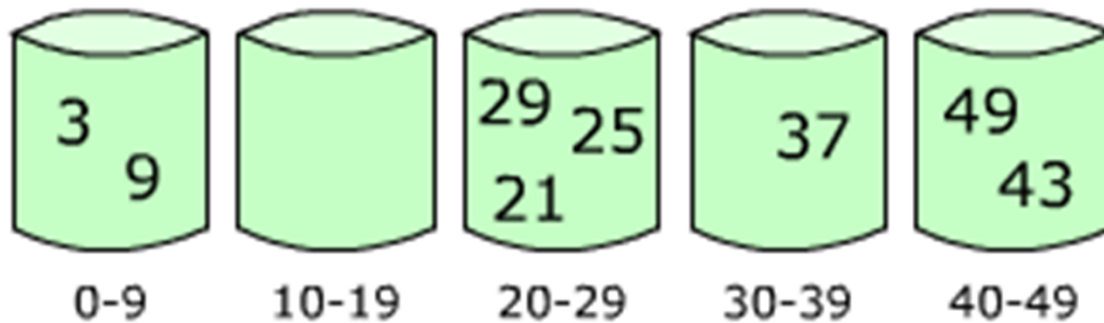
ProxmapSort, или сортиране на Proxmap, е алгоритъм за сортиране, който работи чрез разделяне на масив от елементи с данни или ключове в редица „подмасиви“ (наричани кофи, в подобни видове).

Името е съкратено за изчисляване на „карта на близостта“, която показва за всеки ключ K началото на подмасив, където K ще се намира в крайния сортиран ред. Ключовете се поставят във всеки подмасив чрез сортиране чрез вмъкване.

Ако ключовете са "добре разпределени" между подмасивите, сортирането се извършва в линейно време. Оценките на изчислителната сложност включват броя на подмасивите и използваната функция за картографиране на близост, „ключ на картата“. Това е форма на кофа и радикално сортиране.

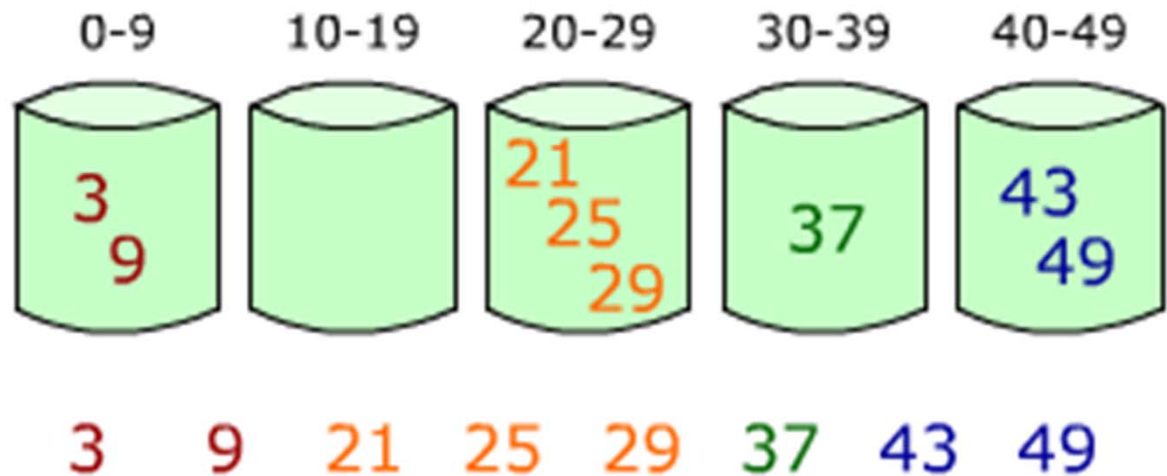
Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)

29 25 3 49 9 37 21 43



Елементите се разпределят между кофите

За разлика от **Bucket sort**, което сортира, след като всички кофи са запълнени, тук елементите се сортират чрез вмъкване, докато се поставят в кофите

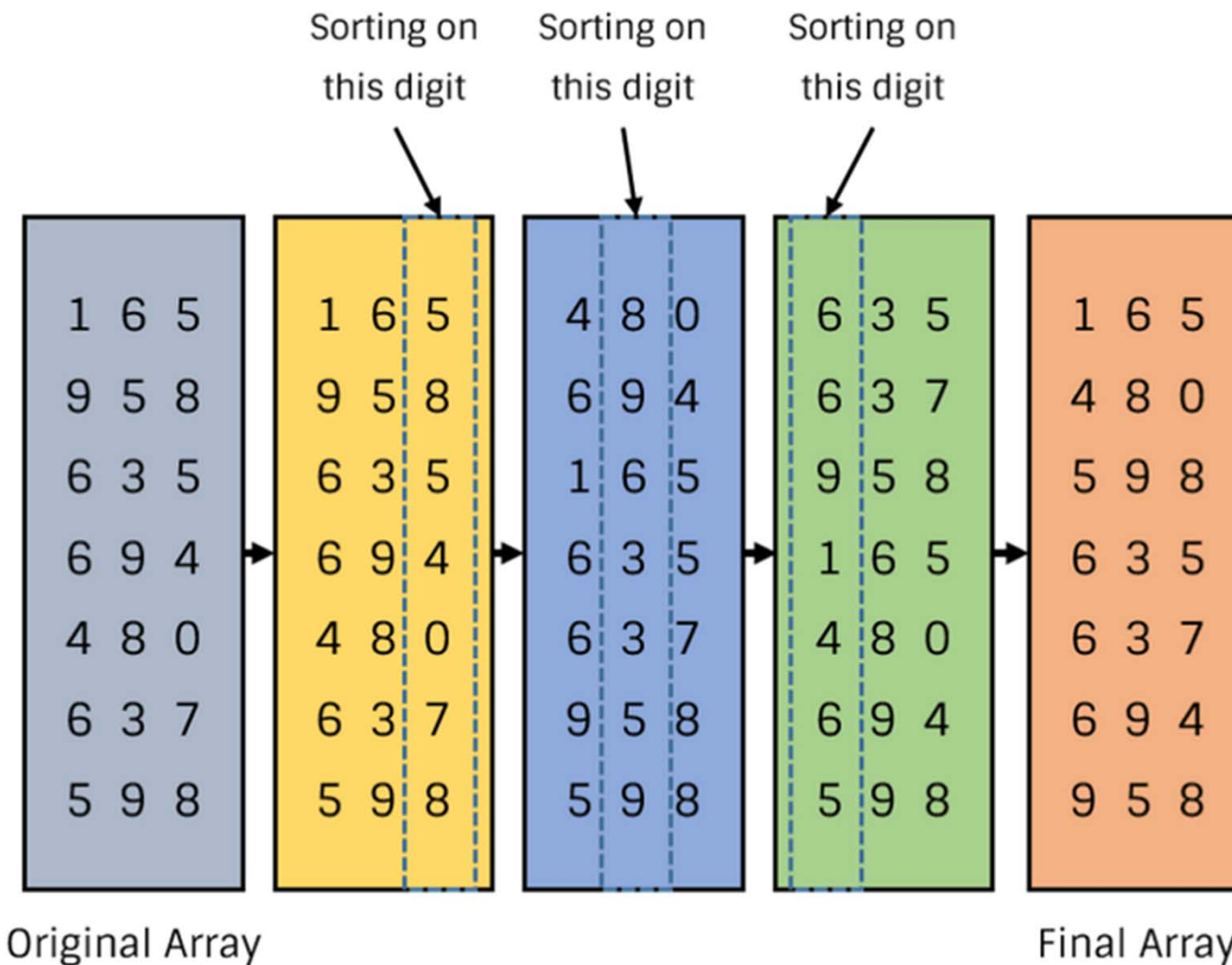


Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)

Radix sort е несравнителен алгоритъм за сортиране. Той избягва сравнението, като създава и разпределя елементи в кофи според техния радикс. За елементи с повече от една значима цифра, този процес на групиране се повтаря за всяка цифра, като се запазва подреждането на предходната стъпка, докато всички цифри бъдат взети предвид. Поради тази причина Radix sort се нарича още цифрово сортиране.

Radix sort може да се приложи към данни, които могат да бъдат сортирани лексикографски, независимо дали са цели числа, думи, перфокарти, карти за игра.

Тема 9 – Алгоритми за сортиране чрез разпределение (*Distribution sorts*)



Radix Sort